# 3. 쇼핑몰 데이터베이스

- **사용자(MEMBER) 테이블의 구조**

| 열이름 | 데이터형 | 정보내용 |
|---|---|---|
| USER_ID | VARCHAR(50) | 사용자 아이디(PK) |
| PASS | VARCHAR(200) | 사용자 비밀번호 |
| USER_NAME | VARCHAR(50) | 사용자 이름 |
| EMAIL | VARCHAR(100) | 사용자 이메일 |
| REG_DATE | DATE | 사용자 등록일 |

- **카테고리(CATEGORY) 테이블의 구조**

| 열이름 | 데이터형 | 정보내용 |
|---|---|---|
| CAT_ID | VARCHAR(50) | 카테고리 아이디(PK) |
| CAT_NAME | VARCHAR(200) | 카테고리 이름 |

- **상품(PRODUCT) 테이블의 구조**

| 열이름 | 데이터형 | 정보내용 |
|---|---|---|
| PROD_ID | INT | 상품 아이디(PK) |
| PROD_NAME | VARCHAR(200) | 상품이름 |
| CAT_ID | VARCHAR(50) | 카테고리 아이디(FK) |
| PRICE | INT | 상품가격 |
| STOCK | INT | 재고수량 |
| REG_DATE | DATE | 상품 등록일 |

- **주문정보(ORDERS) 테이블의 구조**

| 열이름 | 데이터형 | 정보내용 |
|---|---|---|
| ORDER_ID | INT | 주문정보 어이디(PK) |
| USER_ID | VARCHAR(50) | 주문자 아이디(FK) |
| ORDER_DATE | DATE | 주문일자 |
| TOTAL_PRICE | INT | 총 결제금액 |
| STATUS | INT | 주문상태 |

- **주문상품(ORDER_DETAIL) 테이블의 구조**

| 열이름 | 데이터형 | 정보내용 |
|---|---|---|
| DETAIL_ID | INT | 주문상품 아이디(PK) |
| ORDER_ID | INT | 주문정보 아이디(FK) |
| QNT | INT | 주문수량(FK) |
| PRICE | INT | 구매단가 |

- **쇼핑몰 테이블 생성**

사용자 테이블 생성

```
CREATE TABLE MEMBER(
    USER_ID         VARCHAR2(50) NOT NULL PRIMARY KEY,
    PASS            VARCHAR2(200) NOT NULL,
    USER_NAME       VARCHAR2(50) NOT NULL,
    PHONE           VARCHAR2(50),
    ADDR_BASIC      VARCHAR2(200),
    ADDR_DETAIL     VARCHAR2(200),
    EMAIL           VARCHAR2(50),
    REG_DATE        DATE DEFAULT SYSDATE
);
```

---

카테고리 테이블 생성

```
CREATE TABLE CATEGORY(
    CAT_ID          VARCHAR2(20) NOT NULL,
    CAT_NAME        VARCHAR2(200) NOT NULL,
    REG_DAGE        DATE DEFAULT SYSDATE,
    PRIMARY KEY(CAT_ID)
);
```

---

상품 테이블 생성

```
CREATE TABLE PRODUCT(
    PROD_ID         INT NOT NULL PRIMARY KEY,
    CAT_ID          VARCHAR(20) NOT NULL,
    PROD_NAME       VARCHAR2(200) NOT NULL,
    PRICE           INT DEFAULT 0,
    STOCK           INT DEFAULT 100,
    REG_DATE        DATE DEFAULT SYSDATE,
    FOREIGN KEY(CAT_ID) REFERENCES CATEGORY(CAT_ID)
);
```

---

주문정보 테이블 생성

```
CREATE TABLE ORDERS(
    ORDER_ID        INT NOT NULL PRIMARY KEY,
    USER_ID         VARCHAR2(50) NOT NULL,
    RECEIVER        VARCHAR2(50),
    PHONE           VARCHAR2(50),
    ADDR_BASIC      VARCHAR2(200),
    ADDR_DETAIL     VARCHAR2(200),
    TOTAL_PRICE     INT DEFAULT 0,
    ORDER_DATE      DATE DEFAULT SYSDATE,
    STATUS          INT DEFAULT 0,  -- 0: 결제대기중 | 1.결제확인 | 2.배송준비중 | 3.배송중 | 4.배송완료 | 5.주문완료
    FOREIGN KEY(USER_ID) REFERENCES MEMBER(USER_ID)
);
```

---

주문상품 테이블 생성

```
CREATE TABLE ORDER_DETAIL(
    DETAIL_ID       INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    ORDER_ID        INT NOT NULL,
    PROD_ID         INT NOT NULL,
    QNT             INT DEFAULT 1,
    PRICE           INT DEFAULT 0,
    FOREIGN KEY(ORDER_ID) REFERENCES ORDERS(ORDER_ID),
    FOREIGN KEY(PROD_ID) REFERENCES PRODUCT(PROD_ID)
);
```

---

- **쇼핑몰 데이터 입력**

**사용자 테이블 데이터입력**

```
--사용자1
INSERT INTO MEMBER(USER_ID, PASS, USER_NAME, PHONE, ADDR_BASIC, ADDR_DETAIL, EMAIL, REG_DATE)
VALUES('red', 'pass', '김철수', '010-1234-5678', '서울 강남구 테헤란로 123', '101동 502호', 'soo@example.com', SYSDATE-20);
--사용자2
INSERT INTO MEMBER(USER_ID, PASS, USER_NAME, PHONE, ADDR_BASIC, ADDR_DETAIL, EMAIL, REG_DATE)
VALUES('blue', 'pass', '이영희', '010-2345-6789', '경기 성남시 분당구 판교역로 45', '203동 1201호', 'hee@example.com', SYSDATE-19);
--사용자3
INSERT INTO MEMBER(USER_ID, PASS, USER_NAME, PHONE, ADDR_BASIC, ADDR_DETAIL, EMAIL, REG_DATE)
VALUES('green', 'pass', '박민수', '010-3456-7890', '부산 해운대구 해운대로 789', '105동 304호', 'soo@example.com', SYSDATE-17);
```

---

**카테고리 테이블 데이터입력**

```
--카테고리1
INSERT INTO CATEGORY(CAT_ID, CAT_NAME) VALUES('ELEC', '전자제품');
--카테고리2
INSERT INTO CATEGORY(CAT_ID, CAT_NAME) VALUES('CLOTH', '의류');
--카테고리3
INSERT INTO CATEGORY(CAT_ID, CAT_NAME) VALUES('SPORTS', '스포츠/레저');
```

---

**상품 테이블 데이터입력**

```
-- 상품번호 자동 생성을 위한 시퀀스
CREATE SEQUENCE SEQ_PROD_ID START WITH 1001 INCREMENT BY 1;

--1001
INSERT INTO PRODUCT(PROD_ID, PROD_NAME, CAT_ID, PRICE, STOCK)
VALUES(SEQ_PROD_ID.NEXTVAL, '울트라 게이밍 노트북', 'ELEC', 1550000, 50);
--1002
INSERT INTO PRODUCT(PROD_ID, PROD_NAME, CAT_ID, PRICE, STOCK)
VALUES(SEQ_PROD_ID.NEXTVAL, '무선 노이즈캔슬링 헤드폰', 'ELEC', 320000, 120);
--1003
INSERT INTO PRODUCT(PROD_ID, PROD_NAME, CAT_ID, PRICE, STOCK)
VALUES(SEQ_PROD_ID.NEXTVAL, '기계식 키보드(적축)', 'ELEC', 125000, 85);
--1004
INSERT INTO PRODUCT(PROD_ID, PROD_NAME, CAT_ID, PRICE, STOCK)
VALUES(SEQ_PROD_ID.NEXTVAL, '오버핏 기모 후드티', 'CLOTH', 59000, 300);
--1005
INSERT INTO PRODUCT(PROD_ID, PROD_NAME, CAT_ID, PRICE, STOCK)
VALUES(SEQ_PROD_ID.NEXTVAL, '슬림핏 데님 팬츠', 'CLOTH', 45000, 250);
--1006
INSERT INTO PRODUCT(PROD_ID, PROD_NAME, CAT_ID, PRICE, STOCK)
VALUES(SEQ_PROD_ID.NEXTVAL, '스마트 워치 시리즈 9', 'ELEC', 499000, 40);
--1007
INSERT INTO product(prod_id, prod_name, cat_id, price, stock)
VALUES(SEQ_PROD_ID.NEXTVAL, '프리미엄 요가 매트', 'SPORTS', 35000, 100);
--1008
INSERT INTO PRODUCT(PROD_ID, PROD_NAME, CAT_ID, PRICE, STOCK)
VALUES(SEQ_PROD_ID.NEXTVAL, '경량 패딩 조끼', 'CLOTH', 89000, 150);
--1009
INSERT INTO PRODUCT(PROD_ID, PROD_NAME, CAT_ID, PRICE, STOCK)
VALUES(SEQ_PROD_ID.NEXTVAL, '4K 업무용 모니터 27인치', 'ELEC', 380000, 30);
--1010
INSERT INTO PRODUCT(PROD_ID, PROD_NAME, CAT_ID, PRICE, STOCK)
VALUES(SEQ_PROD_ID.NEXTVAL, '단백질 쉐이크 보틀', 'SPORTS', 12000, 500);
```

---

**주문정보 테이블 데이터입력**

```
-- 주문정보 번호 자동 생성을 위한 시퀀스
CREATE SEQUENCE SEQ_ORDER_ID START WITH 20260001 INCREMENT BY 1;

--주문 3건
INSERT INTO ORDERS(ORDER_ID, USER_ID, ORDER_DATE, RECEIVER, PHONE, ADDR_BASIC, ADDR_DETAIL)
VALUES(SEQ_ORDER_ID.NEXTVAL, 'red', SYSDATE-6, '김영희', '010-1234-5678', '서울 강남구 테헤란로 123', '101동 502호');
INSERT INTO ORDERS(ORDER_ID, USER_ID, ORDER_DATE, RECEIVER, PHONE, ADDR_BASIC, ADDR_DETAIL)
VALUES(SEQ_ORDER_ID.NEXTVAL, 'blue', SYSDATE-5, '이혜원', '010-2345-6789', '경기 성남시 분당구 판교역로 45', '203동 1201호');
INSERT INTO ORDERS(ORDER_ID, USER_ID, ORDER_DATE, RECEIVER, PHONE, ADDR_BASIC, ADDR_DETAIL)
VALUES(SEQ_ORDER_ID.NEXTVAL, 'green', SYSDATE-5, '박길동', '010-1234-5678', '서울 강남구 테헤란로 123', '101동 502호');
-- 주문 3건
INSERT INTO ORDERS(ORDER_ID, USER_ID, ORDER_DATE, RECEIVER, PHONE, ADDR_BASIC, ADDR_DETAIL)
VALUES(SEQ_ORDER_ID.NEXTVAL, 'blue', SYSDATE-4, '이철수', '010-2345-6789', '경기 성남시 분당구 판교역로 45', '203동 1201호');
INSERT INTO ORDERS(ORDER_ID, USER_ID, ORDER_DATE, RECEIVER, PHONE, ADDR_BASIC, ADDR_DETAIL)
VALUES(SEQ_ORDER_ID.NEXTVAL, 'red', SYSDATE-4, '김미숙', '010-1234-5678', '서울 강남구 테헤란로 123', '101동 502호');
INSERT INTO ORDERS(ORDER_ID, USER_ID, ORDER_DATE, RECEIVER, PHONE, ADDR_BASIC, ADDR_DETAIL)
VALUES(SEQ_ORDER_ID.NEXTVAL, 'green', SYSDATE-3, '박민수', '010-1234-5678', '서울 강남구 테헤란로 123', '101동 502호');
-- 주문 3건
INSERT INTO ORDERS(ORDER_ID, USER_ID, ORDER_DATE, RECEIVER, PHONE, ADDR_BASIC, ADDR_DETAIL)
VALUES(SEQ_ORDER_ID.NEXTVAL, 'red', SYSDATE-3, '김철수', '010-1234-5678', '서울 강남구 테헤란로 123', '101동 502호');
INSERT INTO ORDERS(ORDER_ID, USER_ID, ORDER_DATE, RECEIVER, PHONE, ADDR_BASIC, ADDR_DETAIL)
VALUES(SEQ_ORDER_ID.NEXTVAL, 'blue', SYSDATE-2, '이영희', '010-2345-6789', '경기 성남시 분당구 판교역로 45', '203동 1201호');
INSERT INTO ORDERS(ORDER_ID, USER_ID, ORDER_DATE, RECEIVER, PHONE, ADDR_BASIC, ADDR_DETAIL)
VALUES(SEQ_ORDER_ID.NEXTVAL, 'green', SYSDATE-1, '박미혜', '010-1234-5678', '서울 강남구 테헤란로 123', '101동 502호');
```

**주문상품 테이블 데이터입력**

```
-- 주문 20260001 (4건)
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260001, 1001, 1, 1550000);
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260001, 1002, 1, 320000);
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260001, 1003, 2, 125000);
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260001, 1007, 2, 35000);
-- 주문 20260002 (3건)
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260002, 1004, 2, 59000);
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260002, 1005, 1, 45000);
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260002, 1008, 1, 89000);
-- 주문 20260003 (2건)
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260003, 1003, 1, 125000);
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260003, 1007, 1, 35000);
-- 주문 20260004 (2건)
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260004, 1002, 1, 320000);
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260004, 1003, 1, 125000);
-- 주문 20260005 (3건)
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260005, 1006, 1, 499000);
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260005, 1003, 1, 125000);
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260005, 1008, 1, 89000);
-- 주문 20260006 (2건)
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260006, 1007, 3, 35000);
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260006, 1005, 1, 45000);
-- 주문 20260007 (1건)
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260007, 1008, 3, 89000);
-- 주문 20260008 (4건)
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260008, 1009, 1, 380000);
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260008, 1001, 1, 1550000);
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260008, 1006, 1, 499000);
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260008, 1010, 1, 12000);
-- 주문 20260009 (3건)
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260009, 1010, 1, 12000);
INSERT INTO ORDER_DETAI(ORDER_ID, PROD_ID, QNT, PRICE)L VALUES (20260009, 1007, 1, 35000);
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260009, 1005, 1, 45000);
```

- **쇼핑몰 데이터베이스 조회**

**1) 상품 테이블의 모든 행, 모든 칼럼의 데이터를 검색한다.**

```
SELECT * FROM PRODUCT;
```

**2) 상품 테이블에서 모든 상품의 상품 아이디, 이름, 가격 칼럼을 검색한다.**

```
SELECT PROD_ID, PROD_NAME, PRICE FROM PRODUCT;
```

**3) 상품 테이블에서 상품들의 카테고리를 중복을 제거하여 검색한다.**

```
SELECT DISTINCT(CAT_ID) FROM PRODUCT;
```

**4) 상품 테이블에서 상품 가격이 50000원 이상이고 100000원 이하인 상품을 검색한다.**

```
SELECT * FROM PRODUCT WHERE PRICE >= 50000 AND PRICE <= 100000;
```

```
SELECT * FROM PRODUCT WHERE PRICE BETWEEN 50000 AND 100000;
```

**5) 상품 테이블에서 카테고리 아이디가 'ELEC'이거나 'SPORTS'인 상품의 모든 칼럼을 검색한다.**

```
SELECT * FROM PRODUCT WHERE CAT_ID='ELEC' OR CAT_ID='SPORTS';
```

```
SELECT * FROM PRODUCT WHERE CAT_ID IN ('ELEC', 'SPORTS');
```

**6) 상품 테이블에서 모든 칼럼과 등록일 포맷을 'YYYY-MM-DD HH24:MI:SS'로 검색한다.**

```
SELECT PRODUCT.*, TO_CHAR(REG_DATE, 'YYYY-MM-DD HH24:MI:SS') FMT_REG_DATE FROM PRODUCT;
```

**7) 상품 테이블에서 '26'년도에 등록한 상품의 모든 칼럼을 검색한다.**

```
SELECT * FROM PRODUCT WHERE TO_CHAR(REG_DATE, 'YY') = '26';
```

**8) 상품 테이블에서 '10'월에 등록하지 않는 상품의 모든 칼럼을 검색한다.**

```
SELECT * FROM PRODUCT WHERE TO_CHAR(REG_DATE, 'MM') != '10';
```

```
SELECT * FROM PRODUCT WHERE TO_CHAR(REG_DATE, 'MM') <> '10';
```

**9) 상품 테이블에서 상품명에 '키보드'가 들어가는 상품의 모든 칼럼을 검색한다.**

```
SELECT * FROM PRODUCT WHERE PROD_NAME LIKE '%키보드%';
```

**10) 집 주소가 '서울'인 사용자들을 검색한다.**

```
SELECT * FROM MEMBER WHERE ADDR_BASIC LIKE '%서울%';
```

**11) 아이디가 'RED'인 사용자 정보를 검색한다.**

```
SELECT * FROM MEMBER WHERE USER_ID = LOWER('RED');
```

```
SELECT * FROM MEMBER WHERE UPPER(USER_ID) = 'RED';
```

- **쇼핑몰 데이터베이스 조인(JOIN)**

1) 새로운 카테고리를 등록한다.

```
INSERT INTO CATEGORY (CAT_ID, CAT_NAME) VALUES ('FOOD', '식품');
```

2) 상품 테이블의 모든 칼럼과 각 카테고리 테이블의 카테고리 이름을 검색한다. (카테고리 이름 '스포츠' 포함)

```
SELECT P.*, CAT_NAME
FROM PRODUCT P
LEFT JOIN CATEGORY C ON P.CAT_ID=C.CAT_ID
ORDER BY PROD_ID;
```

3) 새로운 사용자를 등록한다.

```
INSERT INTO MEMBER(USER_ID, PASS, USER_NAME) VALUES('pink', 'pass', '김하늘');
```

4) 주문정보 테이블의 주문정보와 사용자 테이블의 주문자 이름, 이메일을 검색한다. (주문자 이름 '김철수')

```
SELECT O.ORDER_ID, O.ORDER_DATE, O.USER_ID, M.USER_NAME, M.EMAIL
FROM ORDERS O
LEFT JOIN MEMBER M ON O.USER_ID=M.USER_ID
ORDER BY ORDER_ID;
```

5) 주문정보 테이블의 주문정보와 주문상품 테이블의 상품 아이디, 구매단가를 검색한다.

```
SELECT O.ORDER_ID, O.ORDER_DATE, D.PROD_ID, D.PRICE
FROM ORDERS O
LEFT JOIN ORDER_DETAIL D ON O.ORDER_ID=D.ORDER_ID
ORDER BY O.ORDER_ID;
```

6) 위 검색내용에 상품 테이블의 상품 이름 칼럼을 추가한다. (상품 이름 '헤드폰' 포함)

```
SELECT O.ORDER_ID, O.ORDER_DATE, D.PROD_ID, D.PRICE, P.PROD_NAME
FROM ORDERS O
LEFT JOIN ORDER_DETAIL D ON O.ORDER_ID=D.ORDER_ID
LEFT JOIN PRODUCT P ON D.PROD_ID=P.PROD_ID
ORDER BY O.ORDER_ID;
```

7) 위 검색내용에 사용자 테이블의 주문자 이름 칼럼을 추가한다. (주문자 이름 박민수)

```
SELECT O.ORDER_ID, O.ORDER_DATE, D.PROD_ID, D.PRICE, P.PROD_NAME, M.USER_NAME
FROM ORDERS O
LEFT JOIN ORDER_DETAIL D ON O.ORDER_ID=D.ORDER_ID
LEFT JOIN PRODUCT P ON D.PROD_ID=P.PROD_ID
LEFT JOIN MEMBER M ON O.USER_ID=M.USER_ID
ORDER BY O.ORDER_ID;
```

8) 위 검색내용에 카테고리 테이블의 카테고리 이름 칼럼을 추가한다. (카테고리 이름에 '스포츠' 포함)

```
SELECT O.ORDER_ID, O.ORDER_DATE, D.PROD_ID, D.PRICE, P.PROD_NAME, M.USER_NAME, C.CAT_NAME
FROM ORDERS O
LEFT JOIN ORDER_DETAIL D ON O.ORDER_ID=D.ORDER_ID
LEFT JOIN PRODUCT P ON D.PROD_ID=P.PROD_ID
LEFT JOIN MEMBER M ON O.USER_ID=M.USER_ID
LEFT JOIN CATEGORY C ON P.CAT_ID=C.CAT_ID
ORDER BY O.ORDER_ID;
```

- **쇼핑몰 데이터베이스 집계함수**

**1) 카테고리 테이블에서 카테고리별 상품수를 검색한다. (상품 이름 추가)**

```
SELECT P.CAT_ID, COUNT(*) 상품수
FROM PRODUCT P
GROUP BY P.CAT_ID;
```

```
SELECT P.CAT_ID, CAT_NAME, COUNT(*) 상품수
FROM PRODUCT P
LEFT JOIN CATEGORY C ON P.CAT_ID=C.CAT_ID
GROUP BY P.CAT_ID, CAT_NAME;
```

**2) 주문정보 테이블에서 주문자 아이디별 주문수를 검색한다. (주문자 이름 추가)**

```
SELECT O.USER_ID, COUNT(*) 주문수
FROM ORDERS O
GROUP BY O.USER_ID;
```

```
SELECT O.USER_ID, USER_NAME, COUNT(*) 주문수
FROM ORDERS O
LEFT JOIN MEMBER M ON O.USER_ID=M.USER_ID
GROUP BY O.USER_ID, USER_NAME;
```

**3) 주문상품 테이블에서 상품별 주문상품수, 주문총금액을 검색한다. (상품 이름 추가)**

```
SELECT D.PROD_ID, SUM(QNT) 주문상품수, SUM(QNT*D.PRICE) 주문총금액
FROM ORDER_DETAIL D
GROUP BY D.PROD_ID;
```

```
SELECT D.PROD_ID, PROD_NAME, SUM(QNT) 주문상품수, SUM(QNT*D.PRICE) 주문총금액
FROM ORDER_DETAIL D
LEFT JOIN PRODUCT P ON D.PROD_ID=P.PROD_ID
GROUP BY D.PROD_ID, PROD_NAME;
```

**4) 주문상품 테이블에서 카테고리별 주문상품수, 주문총금액을 검색한다. (카테고리 이름 추가 출력)**

```
SELECT P.CAT_ID, CAT_NAME, SUM(QNT) 주문상품수, SUM(QNT*D.PRICE) 주문총금액
FROM ORDER_DETAIL D
LEFT JOIN PRODUCT P ON P.PROD_ID=D.PROD_ID
LEFT JOIN CATEGORY C ON P.CAT_ID=C.CAT_ID
GROUP BY P.CAT_ID, CAT_NAME;
```

**4) 주문상품 테이블에서 주문일별 주문상품수, 주문총금액을 검색한다.**

```
SELECT TO_CHAR(ORDER_DATE, 'YYYY-MM-DD') 년월, SUM(QNT) 주문상품수, SUM(QNT*D.PRICE) 주문총금액
FROM ORDERS O
LEFT JOIN ORDER_DETAIL D ON O.ORDER_ID=D.ORDER_ID
GROUP BY TO_CHAR(ORDER_DATE, 'YYYY-MM-DD');
```

**5) 주문상품 테이블에서 시도별 주문상품수, 주문총금액을 검색한다.**

```
SELECT SUBSTR(ADDR_BASIC, 1, INSTR(ADDR_BASIC, ' ')) 시도, SUM(QNT) 주문상품수, SUM(D.QNT*PRICE) 주문총금액
FROM ORDERS O
LEFT JOIN ORDER_DETAIL D ON O.ORDER_ID=D.ORDER_ID
GROUP BY SUBSTR(ADDR_BASIC, 1, INSTR(ADDR_BASIC, ' '));
```

- **쇼핑몰 데이터베이스 서브쿼리(SUB QUERY)**

**1) 상품 테이블에서 판매가격이 가장 비싼 상품정보를 검색한다.**

```
SELECT *
FROM PRODUCT
WHERE PRICE  = (SELECT MAX(PRICE) FROM PRODUCT);
```

**2) 상품 테이블에서 카테고리 이름이 '전자제품'인 상품들을 검색한다.**

```
SELECT *
FROM PRODUCT
WHERE CAT_ID=(SELECT CAT_ID FROM CATEGORY WHERE CAT_NAME='전자제품');
```

**3) 주문상품 테이블에서 판매수량이 5개 이하인 상품들을 검색한다.**

```
SELECT *
FROM PRODUCT
WHERE PROD_ID IN (SELECT PROD_ID FROM ORDER_DETAIL GROUP BY PROD_ID HAVING SUM(QNT) < 5);
```

**4) 사용자 테이블의 모든 칼럼과 주문자별 주문횟수를 검색한다.**

```
SELECT M.*,
(SELECT COUNT(*) FROM ORDERS O WHERE M.USER_ID=O.USER_ID) 주문횟수
FROM MEMBER M;
```

**5) 상품 테이블 모든 칼럼과 상품별 주문 수량합계, 주문 총금액을 검색한다.**

```
SELECT P.*,
(SELECT SUM(QNT) FROM ORDER_DETAIL D WHERE P.PROD_ID=D.PROD_ID) 수량합계,
(SELECT SUM(QNT*PRICE) FROM ORDER_DETAIL D WHERE P.PROD_ID=D.PROD_ID) 총금액
FROM PRODUCT P;
```

**6) 카테고리 테이블의 모든 칼럼과 카테고리별 주문 수량합계, 총금액을 검색한다.**

```
SELECT C.*,
(SELECT SUM(QNT) FROM ORDER_DETAIL D JOIN PRODUCT P ON C.CAT_ID=P.CAT_ID GROUP BY C.CAT_ID) 수량합계,
(SELECT SUM(QNT*D.PRICE) FROM ORDER_DETAIL D JOIN PRODUCT P ON C.CAT_ID=P.CAT_ID GROUP BY C.CAT_ID) 총금액
FROM CATEGORY C;
```

**7) 사용자 테이블의 사용자별 모든 칼럼과 주문횟수를 검색한다.**

```
SELECT M.*, T.주문횟수
FROM (SELECT USER_ID, COUNT(*) 주문횟수 FROM ORDERS GROUP BY USER_ID) T
RIGHT JOIN MEMBER M ON T.USER_ID=M.USER_ID;
```

**8) 카테고리 테이블의 카테고리별 모든 칼럼과 상품수량을 검색한다.**

```
SELECT C.*, T.상품수량
FROM (SELECT CAT_ID, COUNT(*) 상품수량 FROM PRODUCT GROUP BY CAT_ID) T
RIGHT JOIN CATEGORY C ON C.CAT_ID=T.CAT_ID;
```

**9) 상품 테이블의 상품별 모든 칼럼과 주문 수량합계, 주문 총금액을 검색한다.**

```
SELECT P.*, T.수량합계, T.총금액
FROM (SELECT PROD_ID, SUM(QNT) 수량합계, SUM(QNT*PRICE) 총금액 FROM ORDER_DETAIL GROUP BY PROD_ID) T
RIGHT JOIN PRODUCT P ON T.PROD_ID=P.PROD_ID;
```

- **쇼핑몰 데이터베이스 갱신(INSERT, UPDATE, DELETE)**

1) 새로운 'BOOK' 카테고리를 추가한다.

```
INSERT INTO CATEGORY(CAT_ID, CAT_NAME) VALUES('BOOK', '도서');
```

2) 새로운 'black' 사용자를 추가한다.

```
INSERT INTO MEMBER(USER_ID, PASS, USER_NAME) VALUES('black', 'pass' , '김블랙');
```

2) 새로운 '자바' 상품을 추가한다.

```
INSERT INTO PRODUCT(PROD_ID, PROD_NAME, CAT_ID, PRICE) VALUES(SEQ_PROD_ID.NEXTVAL, '자바', 'BOOK', '19000');
```

4) 'black' 사용자가 '자바' 상품을 주문한다.

```
INSERT INTO ORDERS(ORDER_ID, USER_ID) VALUES(SEQ_ORDER_ID.NEXTVAL, 'black');
```

```
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, PRICE, QNT) VALUES(SEQ_ORDER_ID.CURRVAL,
(SELECT PROD_ID FROM PRODUCT WHERE PROD_NAME='자바'), (SELECT PRICE FROM PRODUCT WHERE PROD_NAME='자바', 3));
```

5) '김블랙' 사용자의 정보를 수정한다.

```
UPDATE MEMBER
SET PHONE='010-1010-2020', ADDR_BASIC='서울 강서구 화곡동 현대아파트'
WHERE USER_NAME='김블랙';
```

6) 상품 테이블 '도서' 카테고리 상품들의 재고수량을 10% 추가하여 수정한다. (수량은 소수점 첫째자리에서 반올림)

```
UPDATE PRODUCT
SET STOCK = STOCK + ROUND(STOCK * 0.1)
WHERE CAT_ID = (SELECT CAT_ID FROM CATEGORY WHERE CAT_NAME='도서');
```

7) 주문상품 테이블에서 주문 합계액을 계산하여 주문정보 테이블 주문 합계액을 수정한다.

```
UPDATE ORDERS O
SET TOTAL_PRICE = (SELECT SUM(QNT * PRICE) FROM ORDER_DETAIL D WHERE O.ORDER_ID=D.ORDER_ID);
```

8) 주문상품 테이블에서 주문 수량합계를 계산하여 상품 테이블 재고수량을 수정한다.

```
UPDATE PRODUCT P
SET STOCK = STOCK - (SELECT SUM(QNT) FROM ORDER_DETAIL D WHERE P.PROD_ID=D.PROD_ID);
```

9) 주문정보 테이블에서 'black'이 주문한 주문내역을 삭제한다.

```
DELETE FROM ORDER_DETAIL
WHERE ORDER_ID IN (SELECT ORDER_ID FROM ORDERS WHERE USER_ID='black');
```

```
DELETE FROM ORDERS WHERE USER_ID='blacK';
```

10) 카테고리 테이블에서 '도서' 카테고리를 삭제한다.

```
DELETE FROM PRODUCT
WHERE CAT_ID = (SELECT CAT_ID FROM CATEGORY WHERE CAT_NAME='도서');
```

```
DELETE FROM CATEGORY WHERE CAT_NAME='도서';
```

- **쇼핑몰 데이터베이스 테이블 생성, 제거, 변경**

1) 카테고리 이름이 '전자제품'인 상품들을 PRODUCT_ELEC 테이블로 생성한다.

```
CREATE TABLE PRODUCT_ELEC AS
SELECT * FROM PRODUCT WHERE CAT_ID = (SELECT CAT_ID FROM CATEGORY WHERE CAT_NAME='전자제품');
```

2) 위에서 생성한 PRODUCT_ELEC 테이블에 PRIMARY KEY, FOREIGN KEY를 지정한 후 삭제한다.

```
ALTER TABLE PRODUCT_ELEC
ADD CONSTRAINT PK_PRODUCT PRIMARY KEY(PROD_ID);
```

```
ALTER TABLE PRODUCT_ELEC
ADD CONSTRAINT FK_CATEGORY FOREIGN KEY(CAT_ID) REFERENCES CATEGORY(CAT_ID);
```

```
ALTER TABLE PRODUCT_ELEC DROP PRIMARY KEY;
```

```
ALTER TABLE PRODUCT_ELEC DROP CONSTRAINT FK_CATEGORY;
```

3) 배송지가 '서울'인 주문내역을 ORDERS_SEOUL 테이블로 생성한다.

```
CREATE TABLE ORDERS_SEOUL AS
SELECT * FROM ORDERS WHERE ADDR_BASIC LIKE '서울%';
```

4) 위에서 생성한 CATEGORY_ELEC 테이블과 ORDERS_SEOUL 테이블을 제거한다.

```
DROP TABLE CATEGORY_ELEC;
```

```
DROP TABLE ORDERS_SEOUL;
```

5) 카테고리 테이블에 상품수(PROD_CNT)를 저장할 칼럼을 추가하고 상품 테이블에서 상품수를 구하여 수정한다.

```
ALTER TABLE CATEGORY ADD PROD_CNT INT DEFAULT 0;
```

```
UPDATE CATEGORY C
SET PROD_CNT = (SELECT COUNT(*) FROM PRODUCT P WHERE C.CAT_ID=P.CAT_ID);
```

6) 사용자 테이블 이메일 칼럼의 문자길이와 기본값을 변경한다.

```
ALTER TABLE MEMBER MODIFY EMAIL VARCHAR(100) DEFAULT '';
```

7) 카테고리 테이블 'PROD_CNT' 칼럼 이름을 'PROD_COUNT'로 변경한다.

```
ALTER TABLE CATEGORY RENAME COLUMN PROD_CNT TO PROD_COUNT;
```

8) 카테고리 테이블 'PROD_COUNT' 칼럼을 삭제한다.

```
ALTER TABLE CATEGORY DROP COLUMN PROD_COUNT;
```

9) 사용자 테이블에 입력할 때마다 1씩 증가는 칼럼(SEQ_ID)을 추가한다.

```
ALTER TABLE MEMBER ADD SEQ_ID INT GENERATED ALWAYS AS IDENTITY;
```

- **쇼핑몰 데이터베이스 뷰(VIEW)**

**VIEW를 생성하고 생성된 VIEW를 확인하고 생성된 VIEW를 삭제하는 SQL문은 아래와 같다.**

```
CREATE 뷰이름 AS (SELECT 문장);  -- VIEW 생성

SELECT * FROM USER_VIEWS;  -- 생성된 VIEW 확인

DROP VIEW 뷰이름  -- 생성된 VIEW 삭제
```

**1) 상품 테이블 모든 칼럼에 카테고리명을 추가하는 VIEW를 생성한다.**

```
CREATE VIEW VIEW_PROD AS
SELECT P.*, C.CAT_NAME
FROM PRODUCT P
LEFT JOIN CATEGORY C ON P.CAT_ID = C.CAT_ID;
```

**2) 주문정보 테이블 모든 칼럼에 주문자명을 추가하는 VIEW를 생성한다.**

```
CREATE VIEW VIEW_MEMBER AS
SELECT O.*, M.USER_NAME
FROM ORDERS O
LEFT JOIN MEMBER M ON O.USER_ID=M.USER_ID;
```

**3) 주문상품 테이블에 모든 칼럼에 상품명을 추가하는 VIEW를 생성한다.**

```
CREATE VIEW VIEW_DETAIL AS
SELECT D.*, P.PROD_NAME
FROM ORDER_DETAIL D
LEFT JOIN PRODUCT P ON D.PROD_ID=P.PROD_ID;
```

**4) 주문상품 테이블에 모든 칼럼에 상품명, 주문자명을 추가하는 VIEW를 생성한다.**

```
CREATE VIEW VIEW_DETAIL2 AS
SELECT D.*, P.PROD_NAME, M.USER_NAME
FROM ORDER_DETAIL D
LEFT JOIN PRODUCT P ON D.PROD_ID=P.PROD_ID
LEFT JOIN ORDERS O ON D.ORDER_ID=O.ORDER_ID
LEFT JOIN MEMBER M ON M.USER_ID=O.USER_ID;
```

**5) 주문상품 테이블에 모든 칼럼에 상품명, 주문자명, 카테고리명을 추가하는 VIEW를 생성한다.**

```
CREATE VIEW VIEW_DETAIL3 AS
SELECT D.*, P.PROD_NAME, M.USER_NAME, C.CAT_NAME
FROM ORDER_DETAIL D
LEFT JOIN PRODUCT P ON D.PROD_ID=P.PROD_ID
LEFT JOIN ORDERS O ON D.ORDER_ID=O.ORDER_ID
LEFT JOIN MEMBER M ON M.USER_ID=O.USER_ID
LEFT JOIN CATEGORY C ON P.CAT_ID=C.CAT_ID;
```

**6) 사용자 테이블의 모든 칼럼과 사용자별 주문횟수를 추가하는 VIEW를 생성한다.**

```
CREATE VIEW VIEW_MEMBER2 AS
SELECT M.*,(SELECT COUNT(*) FROM ORDERS O WHERE M.USER_ID=O.USER_ID) 주문횟수
FROM MEMBER M;
```

7) 카테고리 테이블의 모든 칼럼과 카테고리별 상품수를 추가하는 VIEW를 생성한다.

```
CREATE VIEW VIEW_PROD2 AS
SELECT C.*, (SELECT COUNT(*) FROM PRODUCT P WHERE C.CAT_ID=P.CAT_ID) 상품수
FROM CATEGORY C;
```

8) 상품 테이블 모든 칼럼에 상품별 주문 수량합계, 총금액을 추가하는 VIEW를 생성한다.

```
CREATE VIEW VIEW_PROD3 AS
SELECT P.*,
(SELECT SUM(QNT) FROM ORDER_DETAIL D WHERE P.PROD_ID=D.PROD_ID) 수량합계,
(SELECT SUM(QNT*PRICE) FROM ORDER_DETAIL D WHERE P.PROD_ID=D.PROD_ID) 총금액
FROM PRODUCT P;
```

9) 카테고리 테이블의 모든 칼럼과 카테고리별 주문 수량합계, 총금액을 추가하는 VIEW를 생성한다.

```
CREATE VIEW VIEW_CATEGORY AS
SELECT C.*,
(SELECT SUM(QNT) FROM ORDER_DETAIL D JOIN PRODUCT P ON D.PROD_ID=P.PROD_ID WHERE C.CAT_ID=P.CAT_ID) 수량합계,
(SELECT SUM(QNT*D.PRICE) FROM ORDER_DETAIL D JOIN PRODUCT P ON D.PROD_ID=P.PROD_ID WHERE C.CAT_ID=P.CAT_ID)
총금액
FROM CATEGORY C;
```

10) 사용자 테이블의 모든 칼럼과 사용자별 주문 수량합계, 총금액을 추가하는 VIEW를 생성한다.

```
CREATE VIEW VIEW_MEMBER3 AS
SELECT M.*,
(SELECT SUM(QNT) FROM ORDER_DETAIL D JOIN ORDERS O ON D.ORDER_ID=O.ORDER_ID WHERE O.USER_ID=M.USER_ID) 수량합계,
(SELECT SUM(QNT*PRICE) FROM ORDER_DETAIL D JOIN ORDERS O ON D.ORDER_ID=O.ORDER_ID WHERE O.USER_ID=M.USER_ID)
총금액
FROM MEMBER M;
```

11) 주문상품 테이블에서 상품별 상품아이디, 상품명, 주문한 수량합계, 총금액을 추가하는 VIEW를 생성한다.

```
CREATE VIEW VIEW_DETAIL4 AS
SELECT D.PROD_ID, P.PROD_NAME, SUM(QNT) 수량합계, SUM(QNT*D.PRICE) 총금액
FROM ORDER_DETAIL D
LEFT JOIN PRODUCT P ON D.PROD_ID=P.PROD_ID
GROUP BY D.PROD_ID, P.PROD_NAME;
```

12) 주문상품 테이블에서 주문일별 주문일, 주문한 수량합계, 총금액을 추가하는 VIEW를 생성한다.

```
CREATE VIEW VIEW_DETAIL5 AS
SELECT ORDER_DATE, SUM(QNT) 수량합계, SUM(QNT*D.PRICE) 총금액
FROM ORDER_DETAIL D
LEFT JOIN ORDERS O ON D.ORDER_ID=O.ORDER_ID
GROUP BY ORDER_DATE;
```

13) 주문상품 테이블에서 사용자 아이디별 사용자 아이디, 사용자명, 주문한 수량합계, 총금액을 추가하는 VIEW를 생성한다.

```
CREATE VIEW VIEW_DETAIL6 AS
SELECT M.USER_ID, M.USER_NAME, SUM(QNT) 수량합계, SUM(QNT*D.PRICE) 총금액
FROM ORDER_DETAIL D
LEFT JOIN ORDERS O ON D.ORDER_ID=O.ORDER_ID
LEFT JOIN MEMBER M ON O.USER_ID=M.USER_ID
GROUP BY M.USER_ID, M.USER_NAME;
```